

PHP - PDO



Bases de données

Introduction

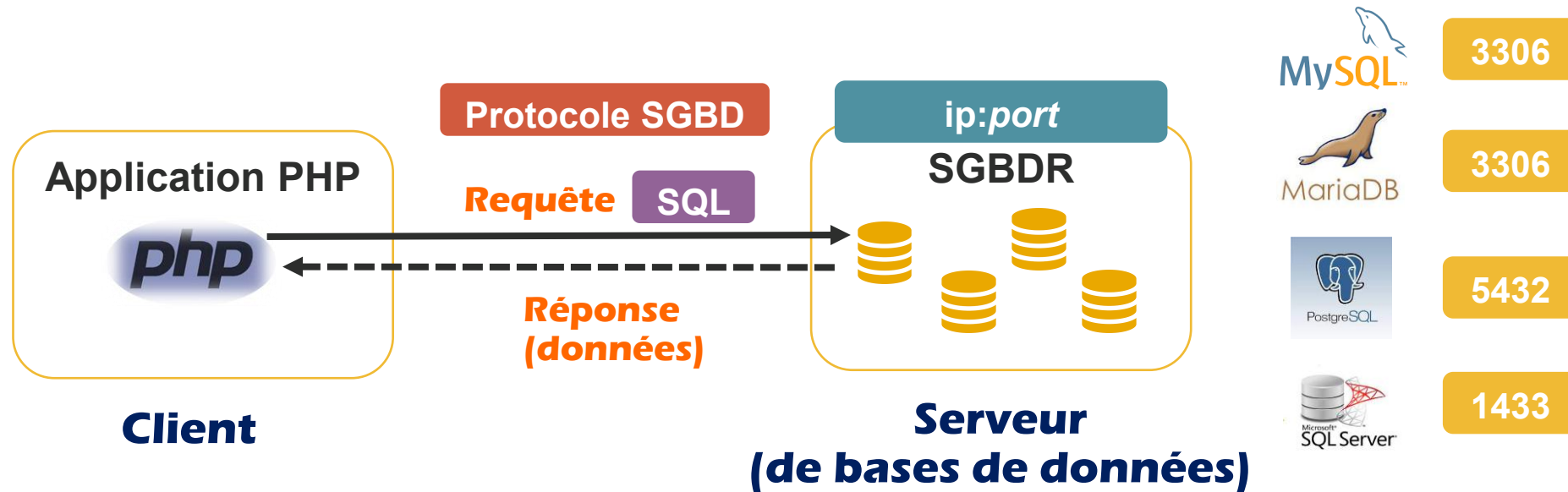




Introduction



Possibilité de se **connecter** à une **base de données relationnelles** depuis une **application PHP**



SGBDR : Système de Gestion de Bases de Données Relationnelles



PHP ne sait pas communiquer directement avec un SGBD



PHP ne connaît pas :

- > le protocole **MySQL**
- > le protocole **PostgreSQL**
- > la manière d'échanger des **requêtes SQL** avec un **SGBD**

> PHP ne parle pas le langage des bases de données

I need help!



Un **driver** est un **composant logiciel** qui permet à un **programme** de **communiquer** avec un autre **logiciel** ou un **matériel** qu'il ne sait pas gérer directement.



Un **driver** est un **traducteur logiciel** qui permet à un **programme** (comme PHP) de **dialoguer** avec un **SGBD** et donc de **parler** le **protocole** du **SGBD**



Il existe un **driver spécifique** pour chaque **SGBD**





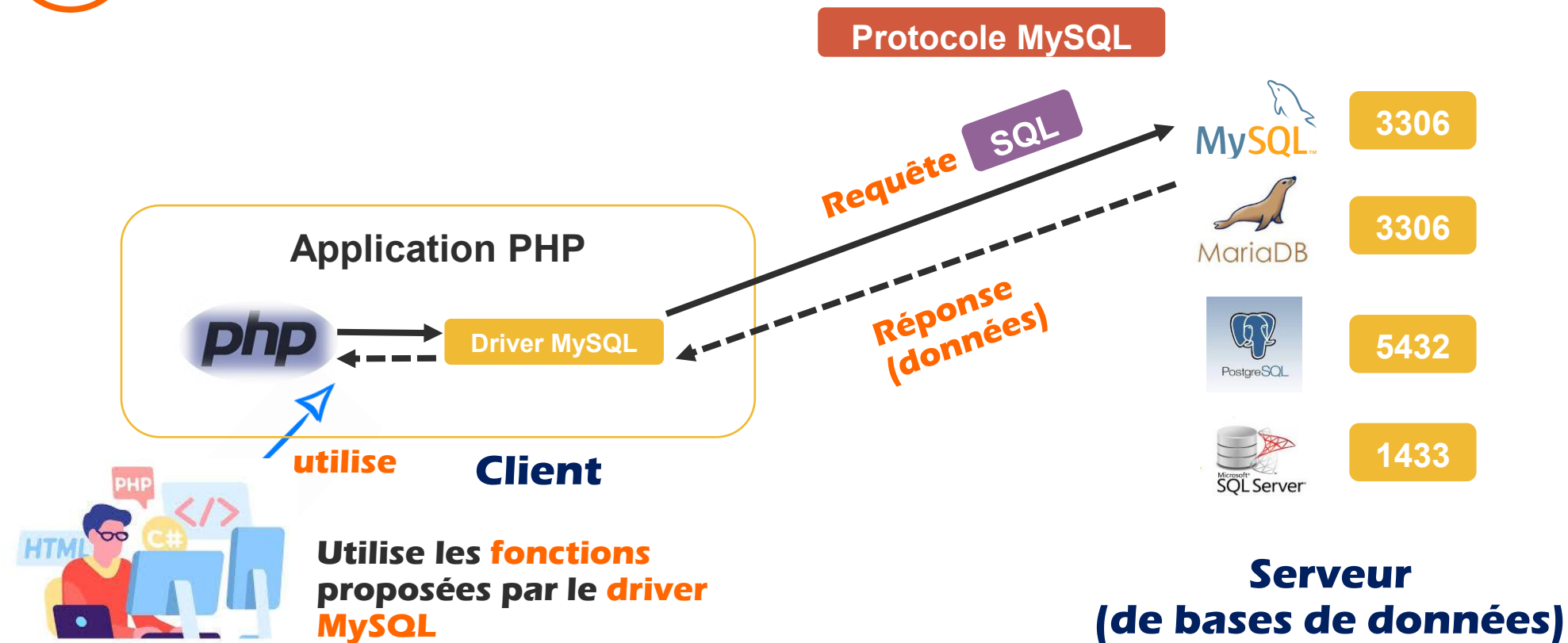
> **Un driver est un ensemble de fonctions PHP déjà fournies, que PHP utilise pour communiquer avec un SGBD** <



Driver



Rôle du **driver**

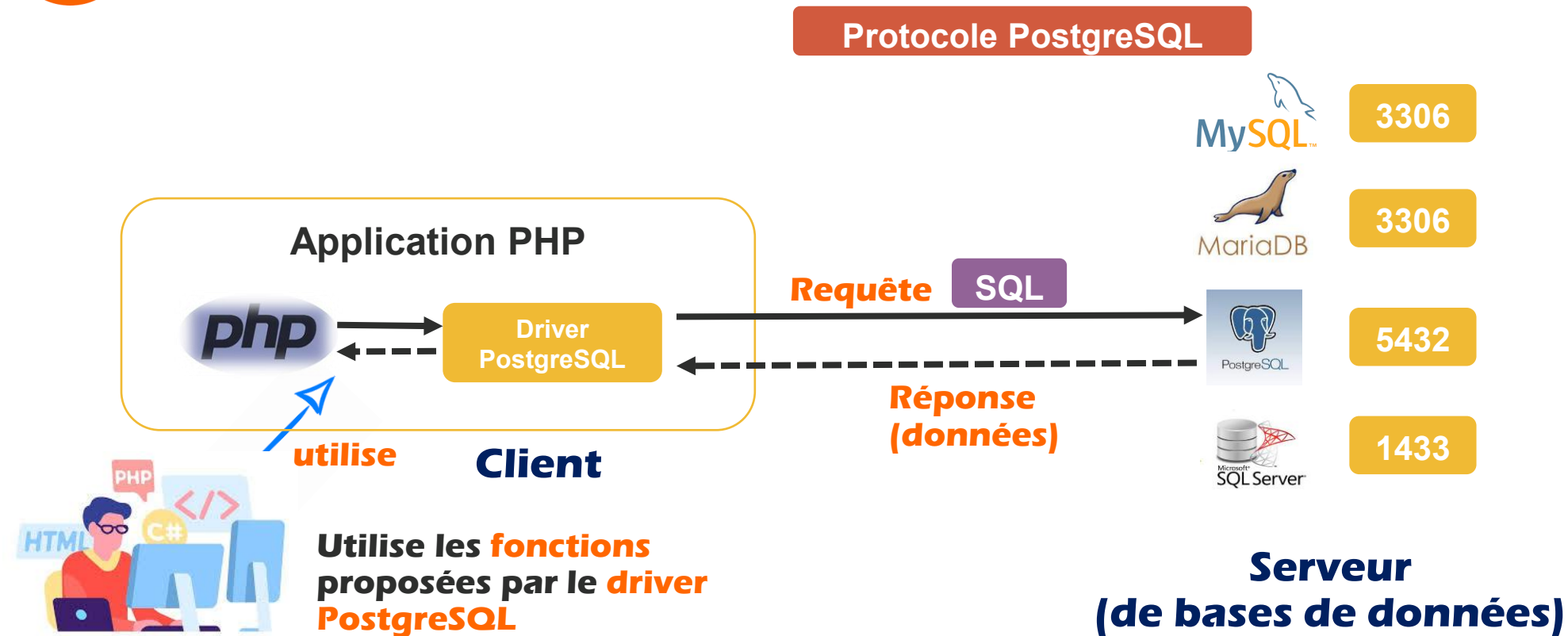




Driver



Rôle du **driver**





Le problème !



- ✓ Simple à comprendre au début
- ✓ Rapide pour un seul SGBD

- ✗ Code dépendant du SGBD
- ✗ Impossible de changer de base facilement
- ✗ **Chaque driver a ses propres fonctions**
- ✗ Pas une bonne pratique professionnelle

👉 **Si on change de MySQL à PostgreSQL**

➡ on **DOIT changer le driver** : tout le code est à

réécrire



➔ **PDO : PHP Data Objects**

➔ **Extension PHP qui permet de COMMUNIQUER avec n'importe quel SGBD depuis une application PHP**



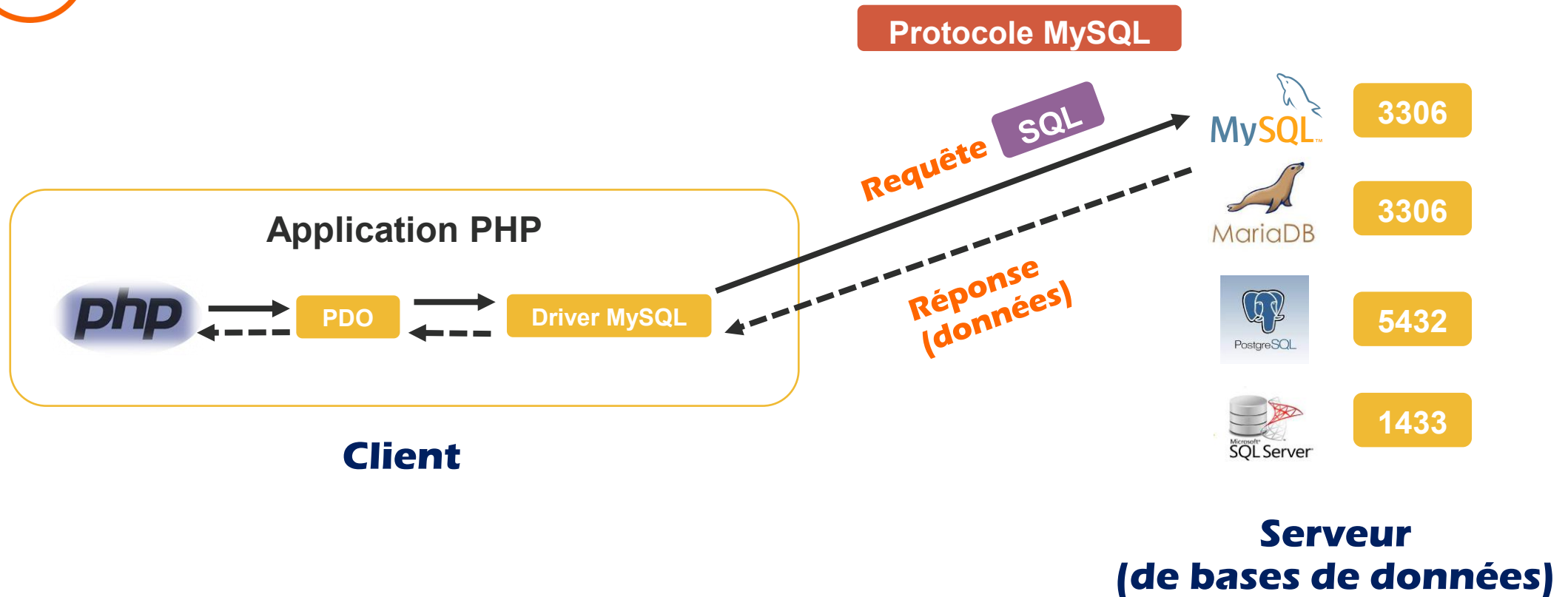
> **PDO est un ensemble de fonctions communes qui permet à une application PHP de se connecter à différents SGBD de la même manière** <



PDO



Rôle de PDO

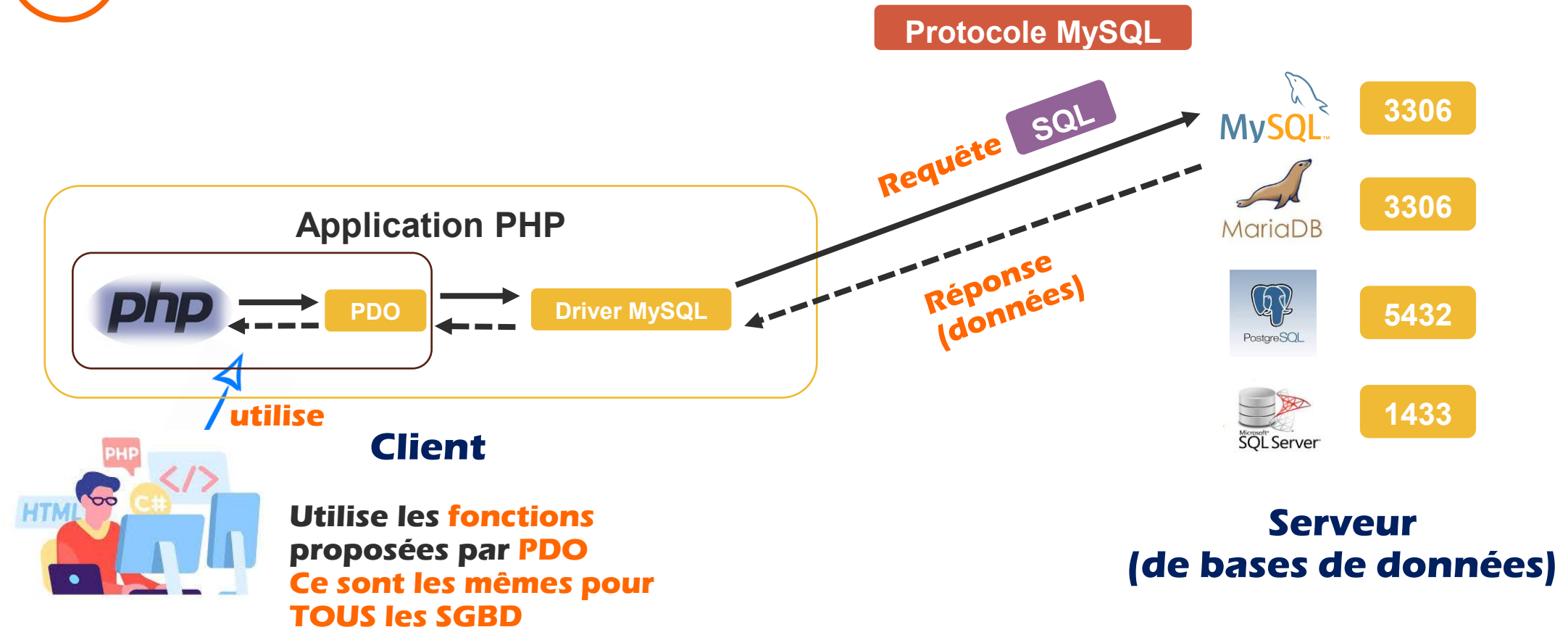




PDO



Rôle de PDO

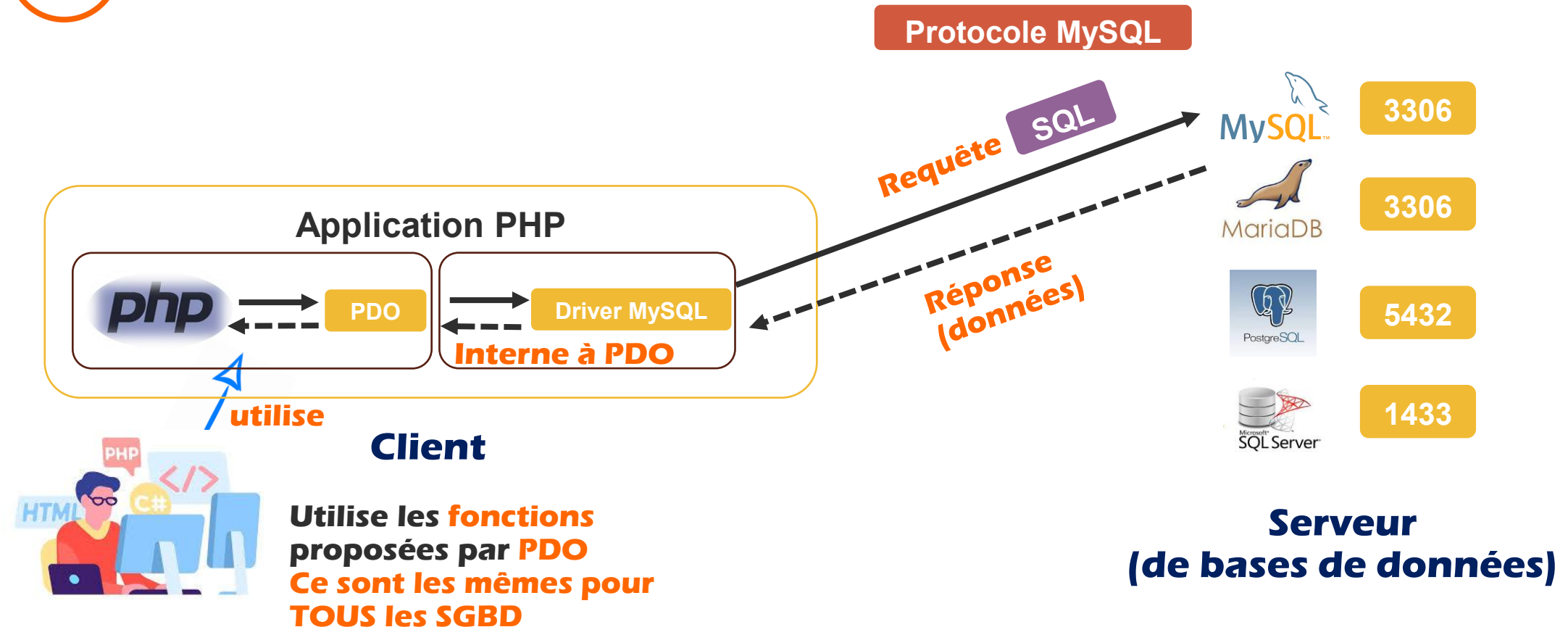




PDO



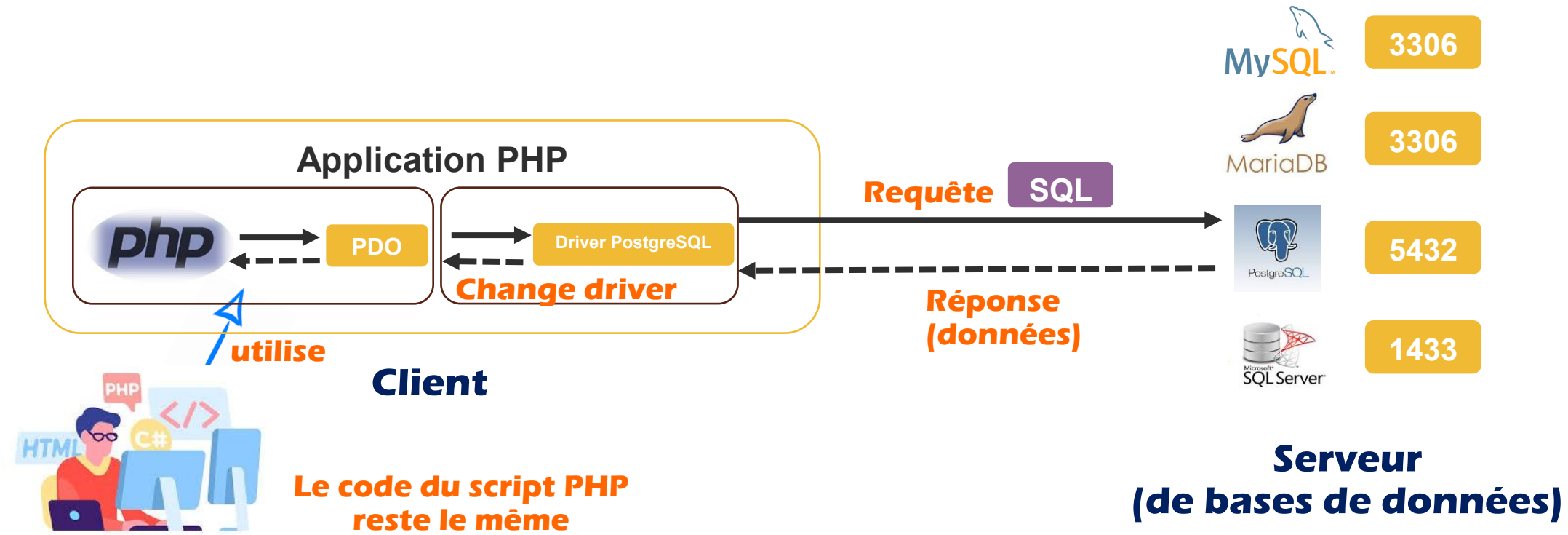
Rôle de PDO





Rôle de **PDO** : changement de SGBD

Protocole PostgreSQL





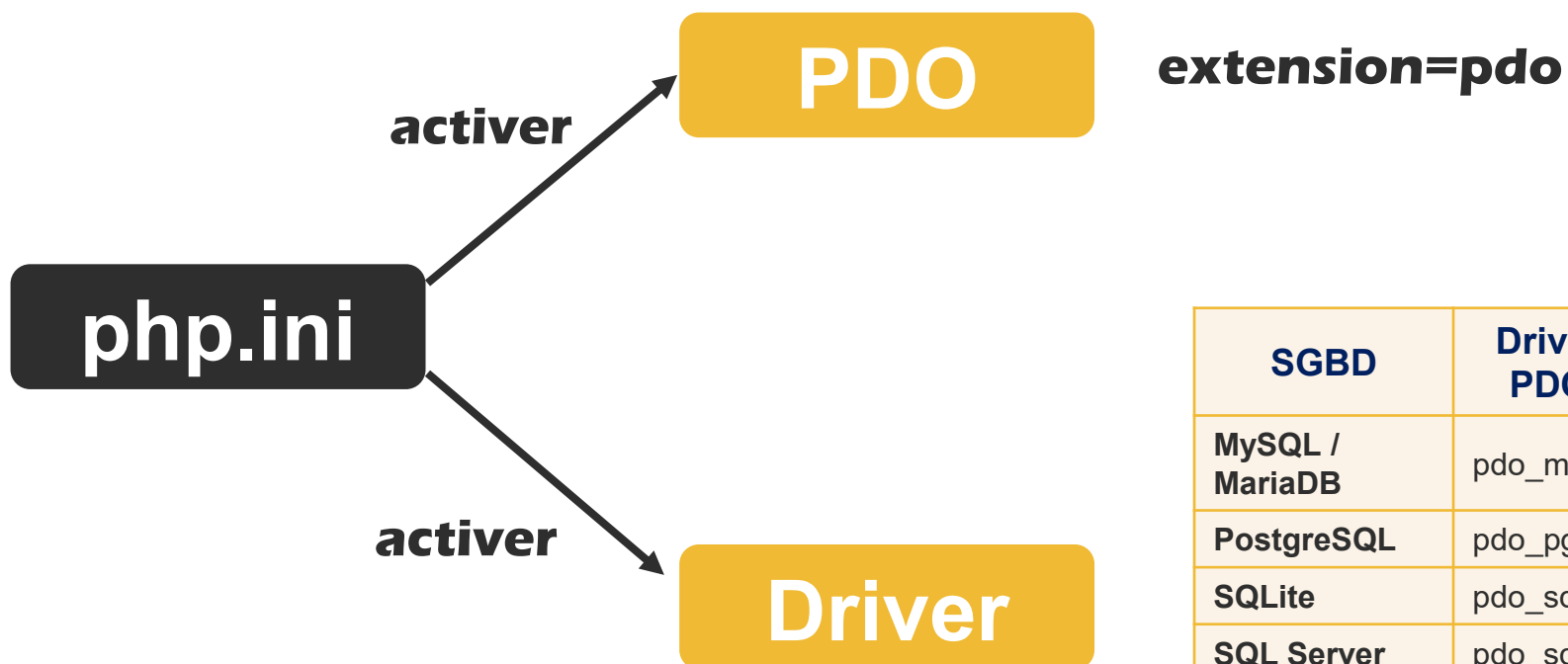
> **Le driver parle la langue
d'un SGBD, PDO parle à
tous les SGBD de la même
façon** <



PDO - Driver



Configuration de PDO - Driver



SGBD	Driver PDO	Utilisation courante
MySQL / MariaDB	pdo_mysql	☀ Le plus utilisé en BTS SIO
PostgreSQL	pdo_pgsql	Très courant en entreprise
SQLite	pdo_sqlite	Bases locales / tests
SQL Server	pdo_sqlsrv	Environnements Microsoft
Oracle	pdo_oci	Grandes entreprises



Préciser à PDO les informations de connexion à la base de données afin d'établir la connexion

DSN

Adresse IP de la machine hôte sur laquelle s'exécute le serveur de base de données

Port sur lequel le serveur est à l'écoute

Nom de la base de données à laquelle on souhaite se connecter

Identifiants de connexion (login + mot de passe)

DSN



Data Source Name



Création de la connexion

```
$host = "...";  
$port = "..."; // Facultatif  
$baseDonnees = "...";  
$utilisateur = "...";  
$motPasse = "...";  
$pdoConnexion = new PDO("mysql:host=$host;dbname=$baseDonnees;charset=utf8",$utilisateur,$motPasse);
```

DSN

connexion

mysql indique à PDO :

« Je veux que tu te connectes à une base MySQL (ou MariaDB) »

👉 PDO comprend alors :

- ✓ qu'il doit utiliser le **driver pdo_mysql**
 - pour **parler le protocole MySQL**
 - et pour **établir la connexion avec le serveur MySQL**



Création de la connexion

```
$host = "...";
$port = "..."; // Facultatif
$baseDonnees = "...";
$utilisateur = "...";
$motPasse = "...";
$pdoConnexion = new PDO("mysql:host=$host;dbname=$baseDonnees;charset=utf8",$utilisateur,$motPasse);
```

DSN

connexion

Chaîne DSN	SGBD correspondant	Driver PDO utilisé
mysql	MySQL / MariaDB	pdo_mysql
pgsql	PostgreSQL	pdo_pgsql
sqlite	SQLite	pdo_sqlite
sqlsrv	SQL Server (Microsoft)	pdo_sqlsrv
oci	Oracle	pdo_oci



Création de la connexion

```
$host = "...";  
$port = "..."; // Facultatif  
$baseDonnees = "...";  
$utilisateur = "...";  
$motPasse = "...";  
$pdoConnexion = new PDO("mysql:host=$host;dbname=$baseDonnees;charset=utf8",$utilisateur,$motPasse);
```

DSN

connexion



Comment tester si la connexion est bien établie ?



Test de la connexion



Utilisation d'un bloc **try ... catch** afin d'**intercepter** les éventuelles **erreurs**

```
try {  
    $pdoConnexion = new PDO("mysql:host=$host;dbname=$baseDonnees;charset=utf8",$utilisateur,$motPasse);  
    $pdoConnexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);  
} catch (PDOException $erreur) {  
    die("Erreur : " . $erreur->getMessage());  
}  
echo "Connexion établie !";
```



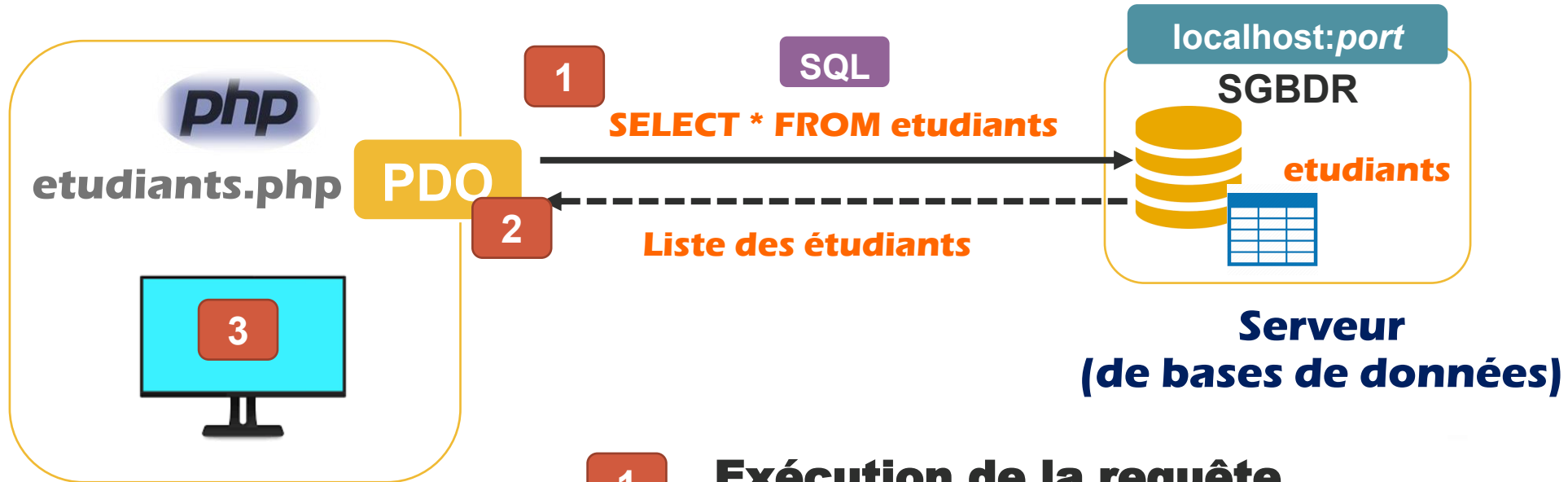
Si il y a une **erreur** dans le bloc **try {...}**, on **l'affiche** et on met **fin au script**



Récupérer la liste des étudiants

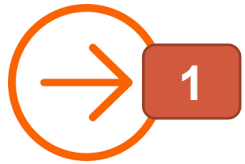


SELECT * FROM etudiants



- 1 **Exécution de la requête**
- 2 **Récupération des résultats**
- 3 **Affichage des résultats**





Exécution de la requête : on indique à **PDO** d'envoyer la requête au serveur afin qu'il l'exécute

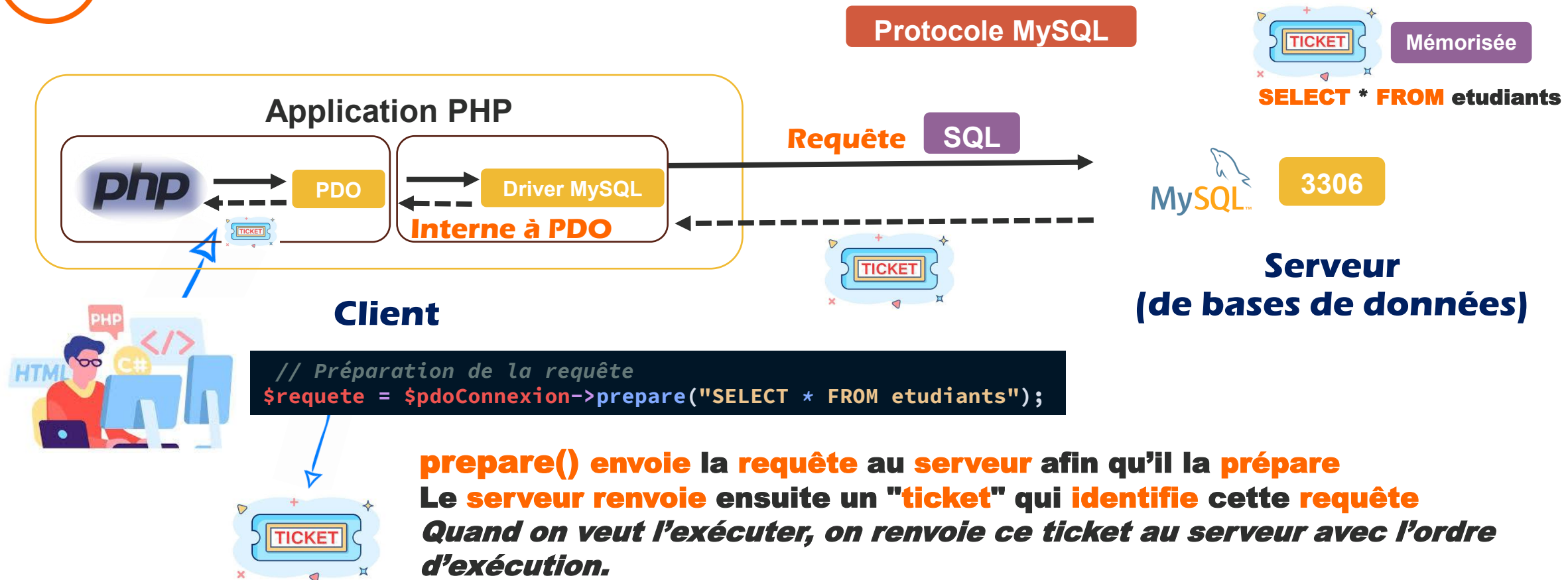
- > **Préparation de la requête**
- > **Exécution de la requête**

```
// Préparation de la requête
$requete = $pdoConnexion->prepare("SELECT * FROM etudiants");

// Exécution de la requête
$requete->execute();
```



➔ 1 Préparation de la requête





Préparation de la requête

- > Concrètement, que fait le **SGBD** quand il "**prépare**" une **requête** ?
- > Quand le **serveur reçoit** une **requête** préparée, il :

1.Vérifie la syntaxe SQL

👉 la requête est-elle bien écrite ?

2.Analyse la requête

👉 quelles tables ? quelles colonnes ? quelles conditions ?

3.Construit un plan d'exécution

👉 comment accéder aux données ?

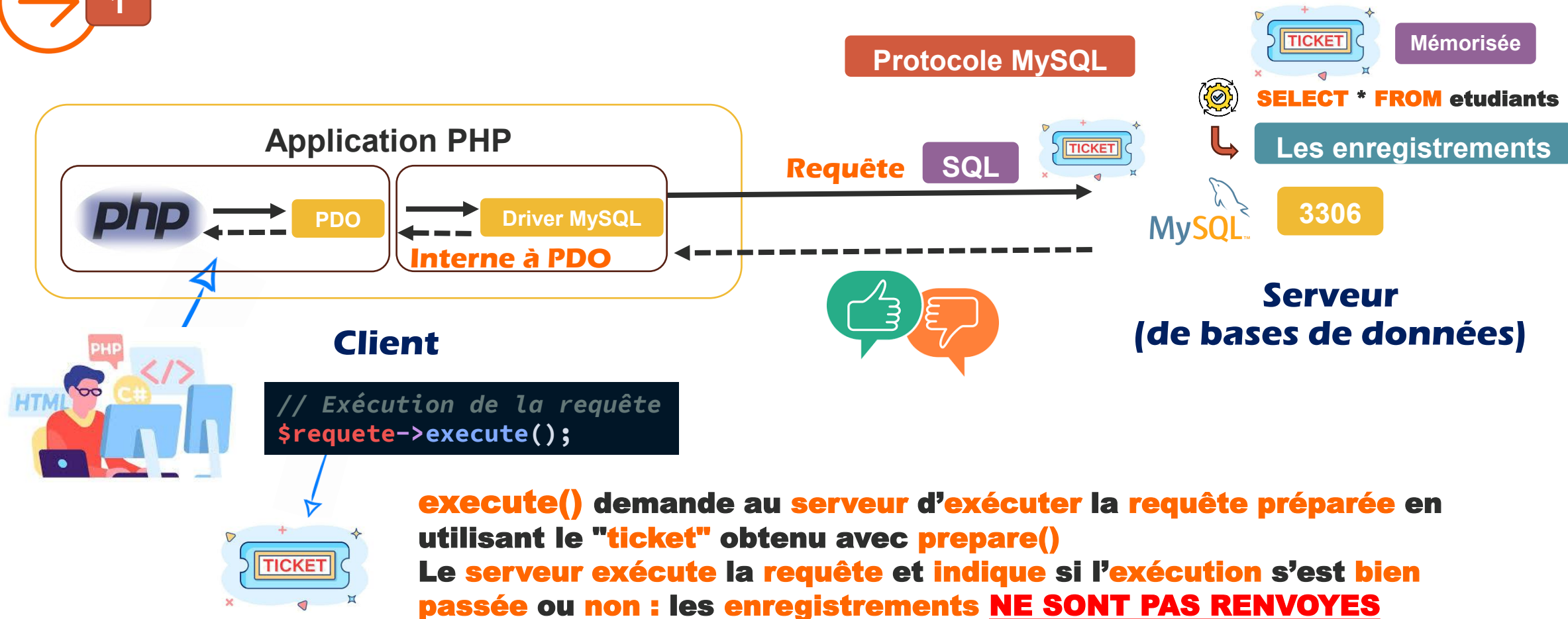
4.Réserve un identifiant interne : le "**ticket**"

👉 pour reconnaître cette requête plus tard

- > La **requête** est **prête**, mais **pas encore exécutée**



➔ 1 Exécution de la requête





Récupération des résultats : on indique sous quelle forme on souhaite récupérer les résultats

```
// On indique sous quelle forme récupérer les résultats  
// ICI tableau associatif  
$requete->setFetchMode(PDO::FETCH_ASSOC);  
  
// Récupération dans résultats  
$etudiants = $requete->fetchAll();
```

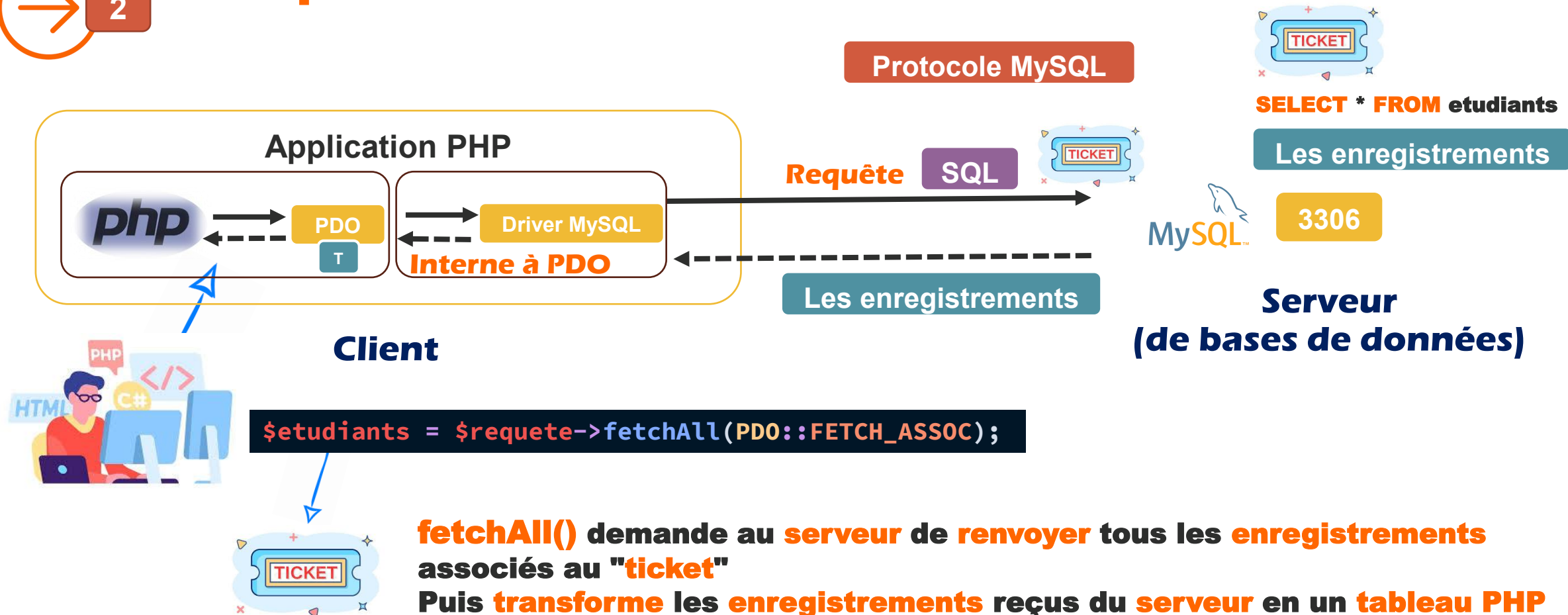
```
$etudiants = $requete->fetchAll(PDO::FETCH_ASSOC);
```

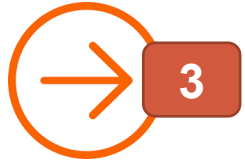


Requête de sélection



➔ 2 Récupération des résultats





Affichage des résultats : on parcourt les données afin de les présenter à l'utilisateur

```
// Affichage des résultats
// Parcours du tableau $etudiants
foreach ($etudiants as $etudiant) {
    echo $etudiant['id_etudiant'] . " "
        . $etudiant["prenom_etudiant"] . " "
        . $etudiant["nom_etudiant"] . " "
        . $etudiant['email_etudiant']
        . PHP_EOL;
}
```



Requête de sélection



Le code complet

```
<?php
// Informations de connexion à la base de données
$host = "...";
$baseDonnees = "...";
$utilisateur = "...";
$motPasse = "...";

// Connexion à la base de données
try {
    $pdoConnexion = new PDO("mysql:host=$host;dbname=$baseDonnees;charset=utf8",$utilisateur,$motPasse);
    $pdoConnexion->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $erreur) {
    die("Erreur : " . $erreur->getMessage());
}

// Préparation de la requête
$requete = $pdoConnexion->prepare("SELECT * FROM etudiants");
// Exécution de la requête
$requete->execute();

// Récupération des résultats dans un tableau associatif
$requete->setFetchMode(PDO::FETCH_ASSOC);
$etudiants = $requete->fetchAll();

// Affichage des résultats
foreach ($etudiants as $etudiant) {
    echo $etudiant['id_etudiant'] . " " . $etudiant['prenom_etudiant'] . " "
        . $etudiant['nom_etudiant'] . " " . $etudiant['email_etudiant'] . PHP_EOL;
}
```